TOPIC:
CLASS IMBALANCE AND FEW-SHOT LEARNING WITH OBJECT DETECTION ALGORITHM YOLO3

Table of Contents

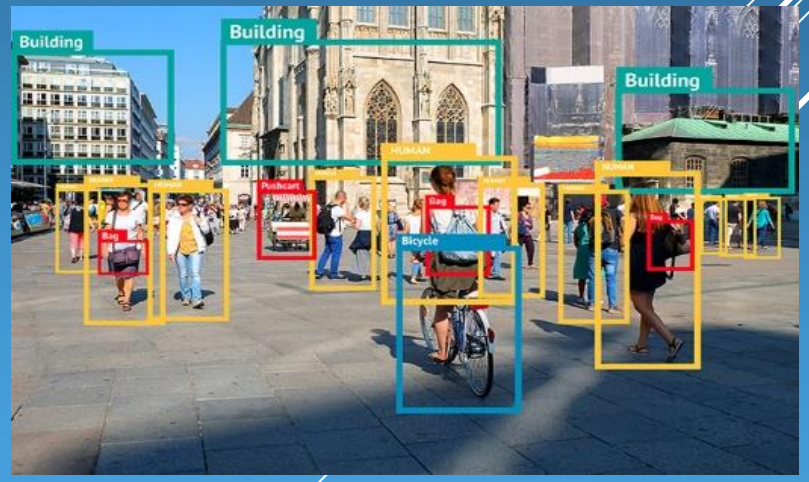
1. Background

Computer Vision:

- An area of artificial intelligence which enable computers to derive information from images
- Two essential technologies: deep learning and convolutional neural network (CNN)

Object detection:

- A branch of computer vision: identify object types and their locations in images
- Highly practical fields: autonomous driving, video surveillance, anomaly detection
- Emerged since 2000 due to deep learning and power of GPU



1. Background

Object Detection

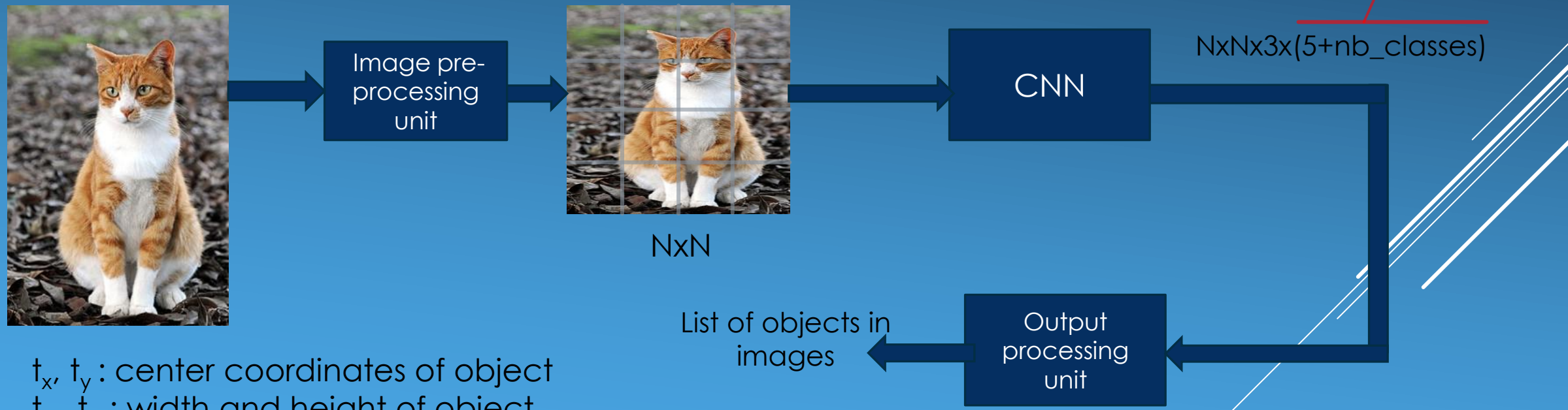
- Two categories of algorithms:
  + One stage: YOLO , SSD (2016), RetinaNet(2017), YOLO3 (2018), YOLO5(2020), YOLOR(2021)
  + Two stages: RCNN, Fast RCNN, Faster RCNN

YOLO3 developed by Joseph Redmon in 2018: Main focus of this dissertation together with class imbalance and few-shot learning.

2. YOLO algorithms details

Images are divided into NxN grid then pass through a CNN network. This network will detect objects in each cell

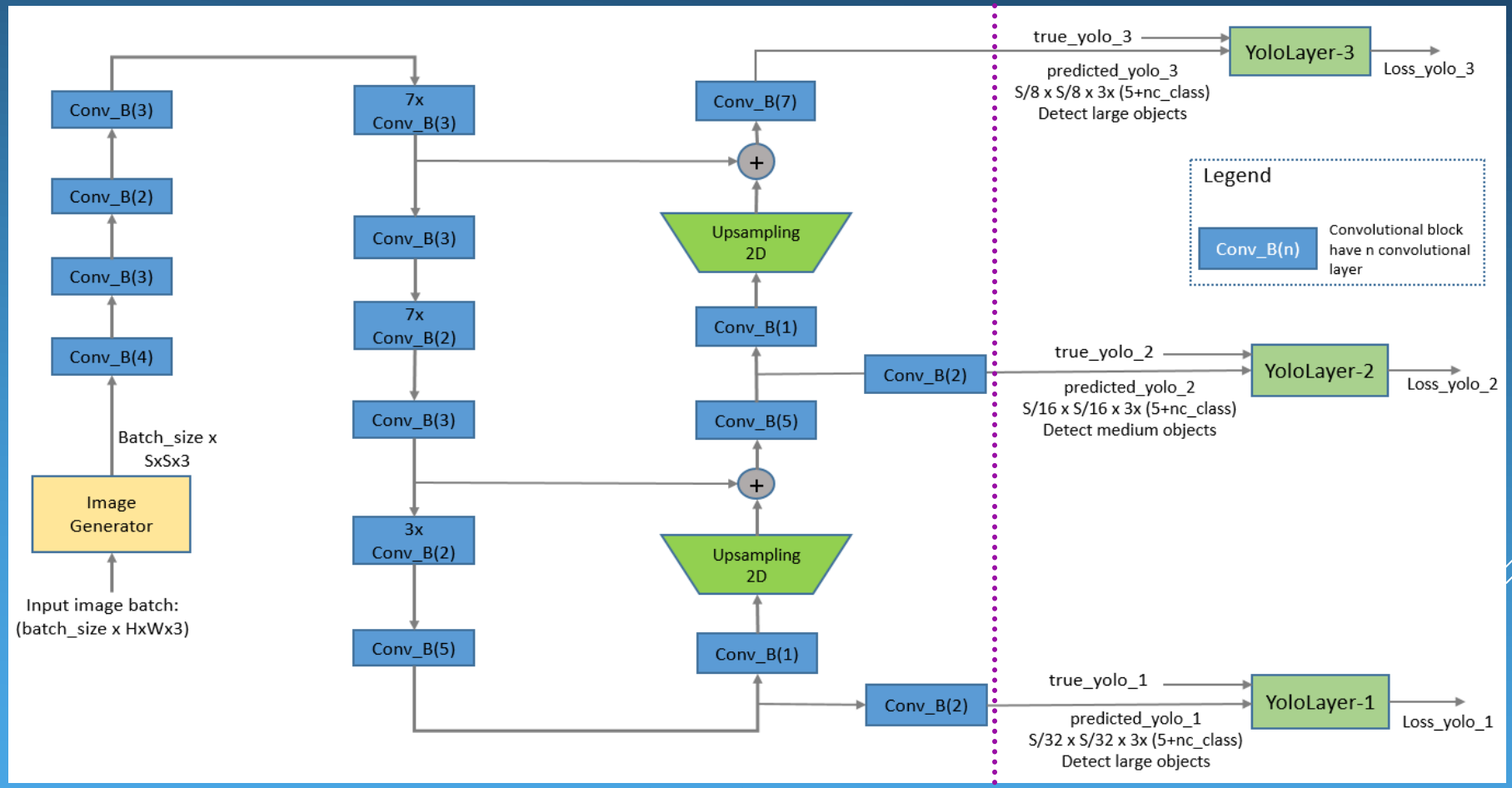
- Each cell go through CNN will output three vector objects : (t_x, t_y, t_w, t_h, p_c , C_1,...,C_k)



t_x, t_y : center coordinates of object
t_w, t_h : width and height of object
P_c : probability that an object exist in the cell
C_1, C_2,..., C_k: probability that detected object is class 1,..., class k

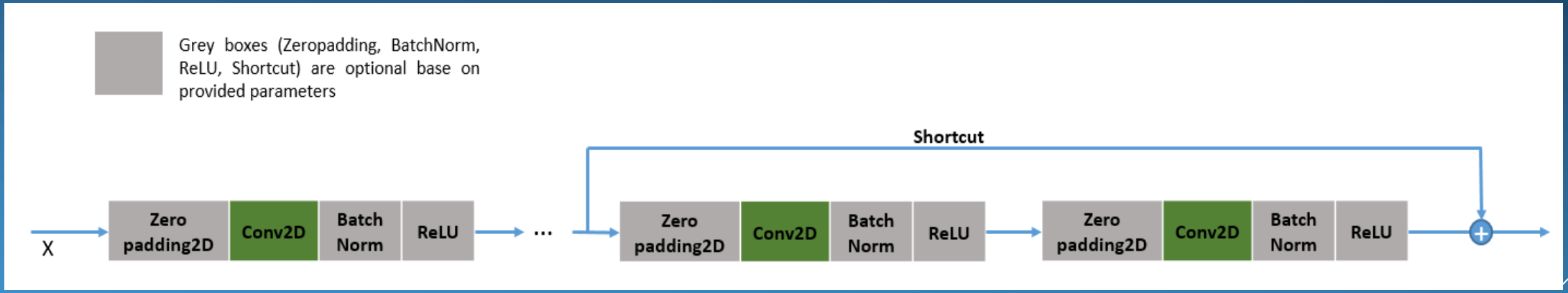
2. YOLO algorithms details

Details of image pre-processing unit and CNN

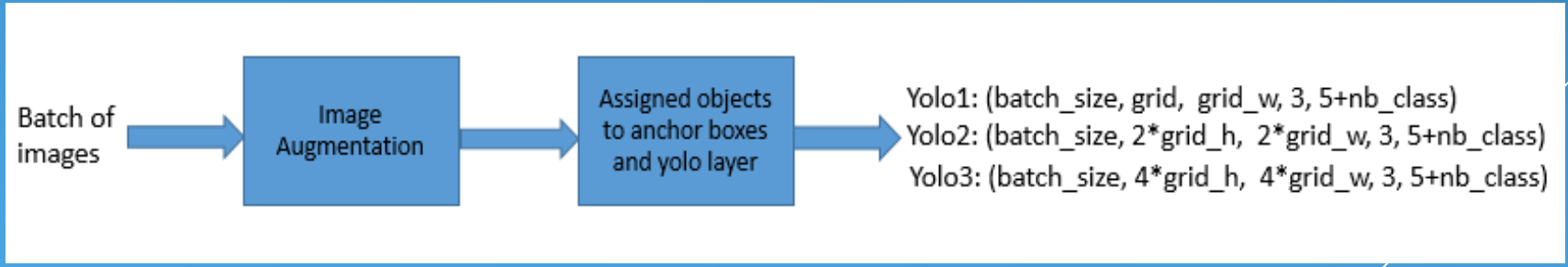


2. YOLO algorithms details

Details of image pre-processing unit and CNN
Convolutional Block details:

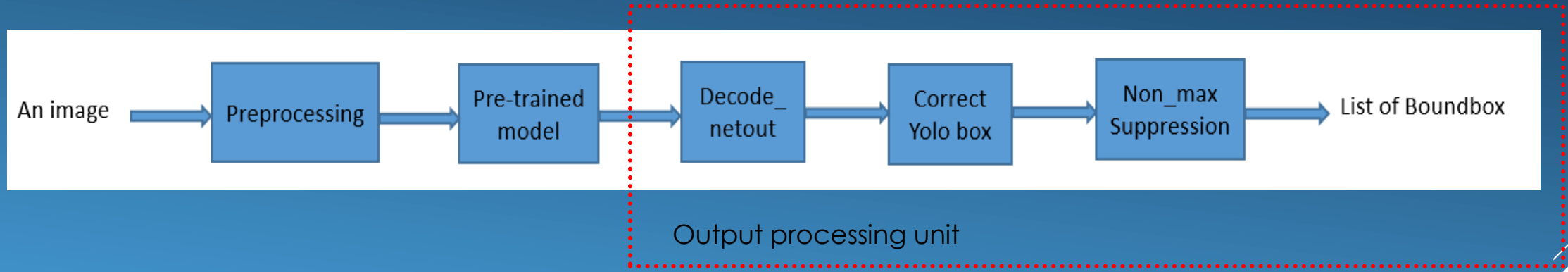


Generator unit: Adjust size of image, augment images and assign objects to YOLO layer



2. YOLO algorithms details

Output processing unit details:

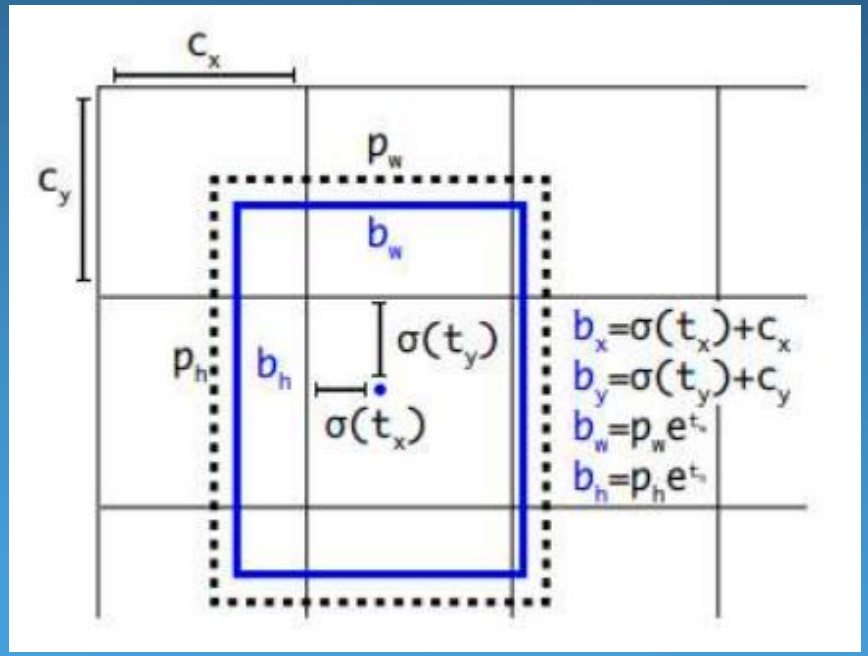


Decode_netout:
- Calculating objectness score
- Removing boxes with objectness score which is less than 0.5
- Returning a list of boundbox object (x_{min}, y_{min}, x_{max}, y_{max}, objectness, label)

Non_max Suppression:
Removing duplicated/overlap bounding boxes to ensure that each object is detected only one.

2. YOLO algorithms details

Relations between vector (t_x, t_y, t_w, t_h) and coordinates of objects are represented by following formulas:



NxN grid

$$b_x = б(t_x) + c_x$$

$$b_y = б(t_y) + c_y$$

$$b_w = p_w\, e^{t_w}$$

$$b_h = p_h\, e^{t_h}$$

- b_x, b_y : centered coordinates of bounding box on (0, N) scale
- b_w, b_h: width and height of bounding box on standardized image scale.
- p_w, p_h: width and height of assigned anchor box
- б: sigmoid function

sigmoid(x)= 1/(1+exp(-x))

2. YOLO algorithms details

Loss function has three components: Classification loss, Regression Loss, Confidence Loss

- Classification Loss penalize for incorrect classification:

$$L\_class = -\lambda_{class} \sum_{i=0}^{N^2} \sum_{j=0}^{B} I_{ij} \log(p_{groundtruth\_class})$$

- Regression loss penalize for the difference between predicted coordinate and actual coordinate of objects

$$L_{regression} = \lambda_{regression} \{ \sum_{i=0}^{N^2} \sum_{j=0}^{B} I_{ij} [(b_{x\_ij} - \widehat{b_{x\_ij}})^2 + (b_{y\_ij} - \widehat{b_{y\_ij}})^2] +$$

$$\sum_{i}^{N^2} \sum_{j=0}^{B} I_{ij} [(t_{w\_ij} - \widehat{t_{w\_ij}})^2 + (t_{h\_ij} - \widehat{t_{h\_ij}})^2] \} * (2 - box_{area_{ij}})^2$$

2. YOLO algorithms details

- Confidence loss penalizes for difference between predicted and actual confidence(actual confidence is one when an object is present, otherwise zero)

$$L\_confidence = \sum_{i=1}^{N^2} \sum_{j=0}^{B} [I_{ij}(p_{c\_ij} - \hat{p_{c\_ij}}) * \lambda_{obj} + (1 - I_{ij}) * D_{ij} * \lambda_{no\_obj}]^2$$

- Total loss:

$$L = L\_class + L\_regression + L\_confidence$$

N, B: grid size and number of anchor boxes per cell, B =3 for YOLO3
b_{x_ij}, b_{y_ij}: center coordinates of bounding box at cell i for anchor box j
t_{x_ij}, t_{y_ij}: the width and height of the bounding box at cell I, anchor box j
I_{ij}: equal to 1 if an object exist in cell I and anchor j, otherwise equal to 0
$p_{groundtruth}$: predicted probability of groundtruth label

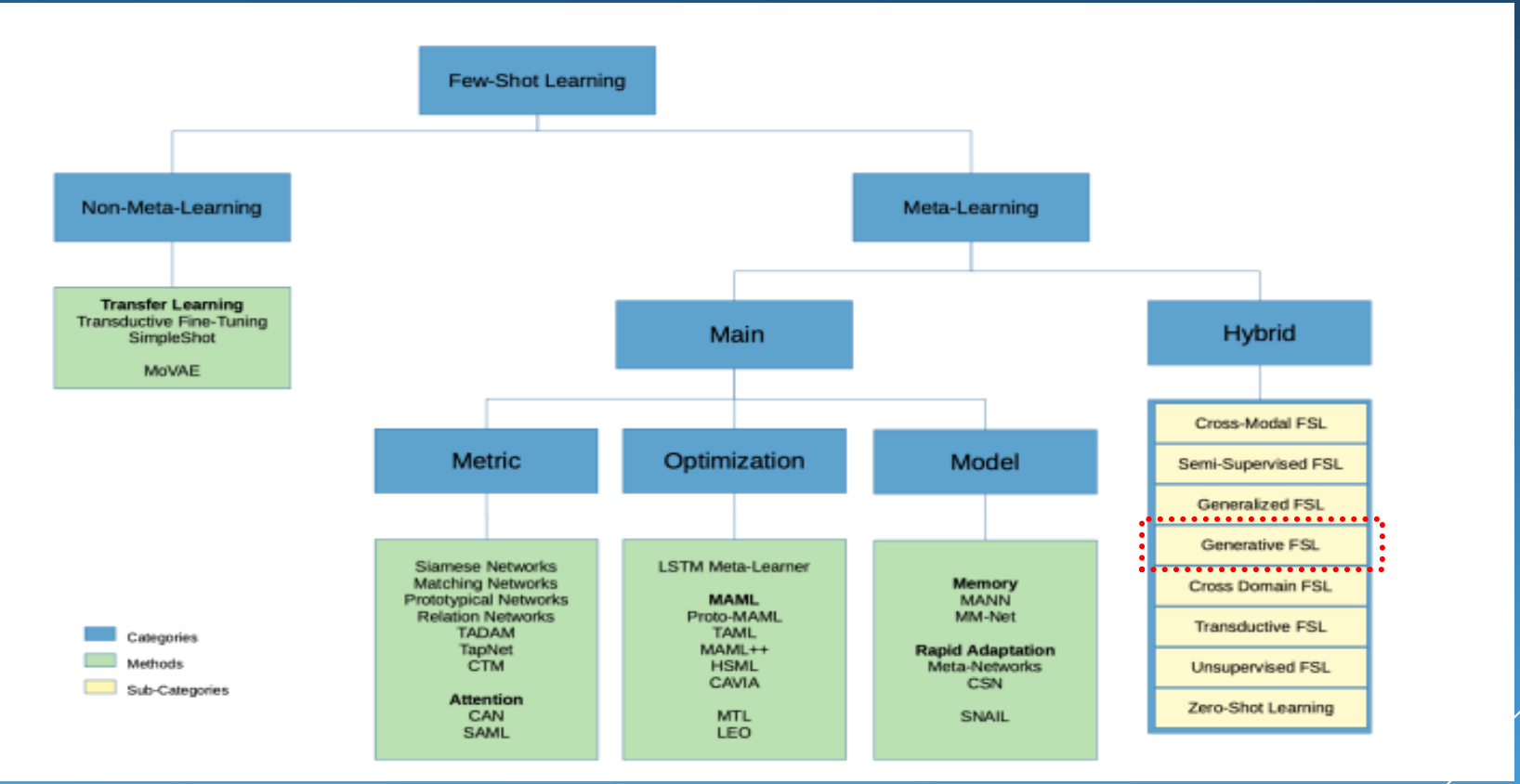
3. Data, Class Imbalance and Few-Shot learning

In machine learning and computer vision both data quality and quantity has impacts to performance of models. We investigate two common issues: class imbalance and few-shot learning:

- Class imbalance:
  - Some classes have dominant representation in the dataset while others have much lower.
  - Solution: collecting more data for under-represented classes, down-sampling, up-sampling, generate synthetic data
- Few-shot learning:
  - Situations that we have only a limited number of samples for training
  - Reason: It is difficult to collect more data or it is high cost to gather data
  - More popular in classification problems but object detection also get the attention recently

3. Data, Class Imbalance and Few-Shot learning

Approaches to few-shot learning

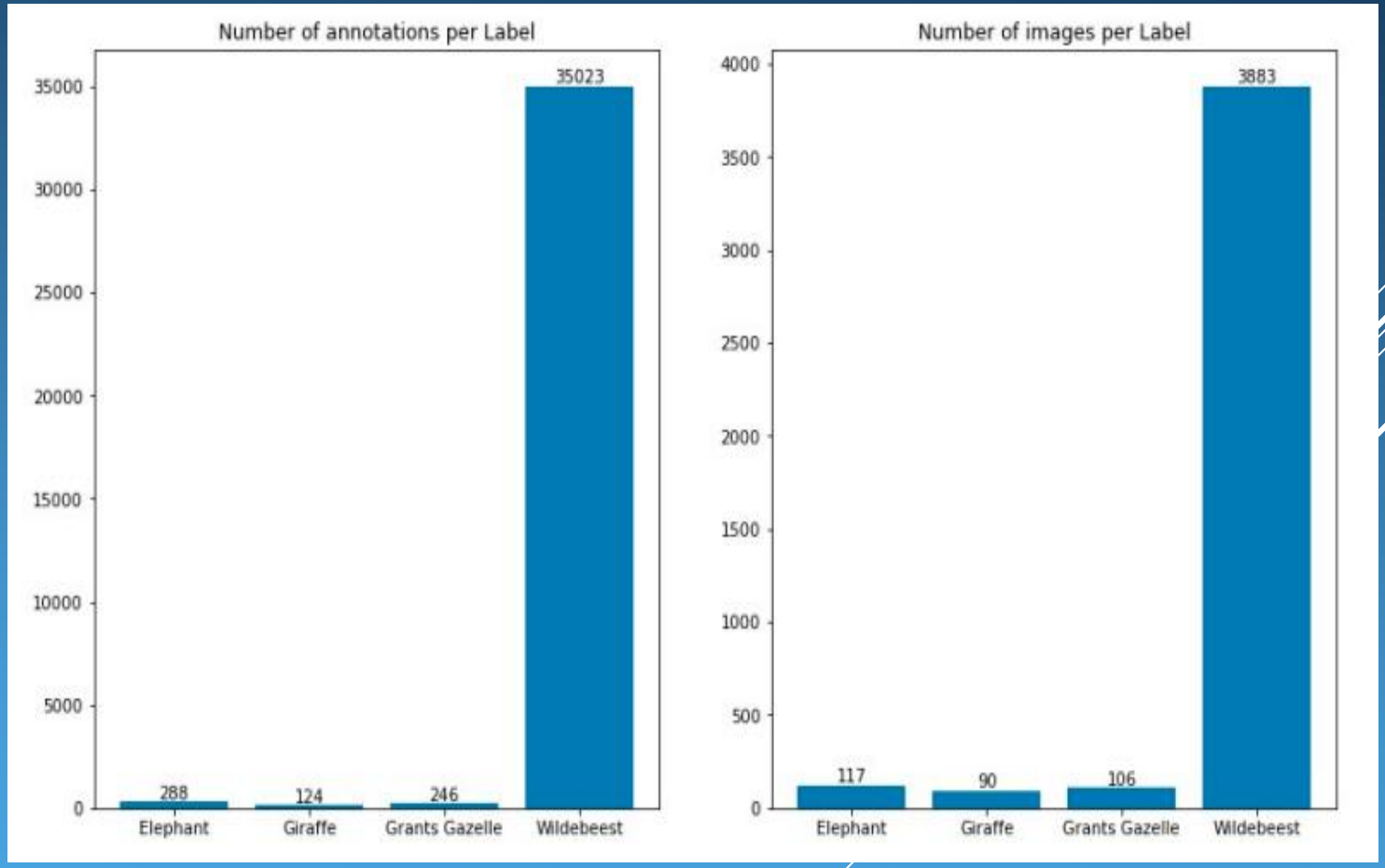


We stick with YOLO3 then "Generative FSL" will be used in this project.

3. Data, Class Imbalance and Few-Shot learning

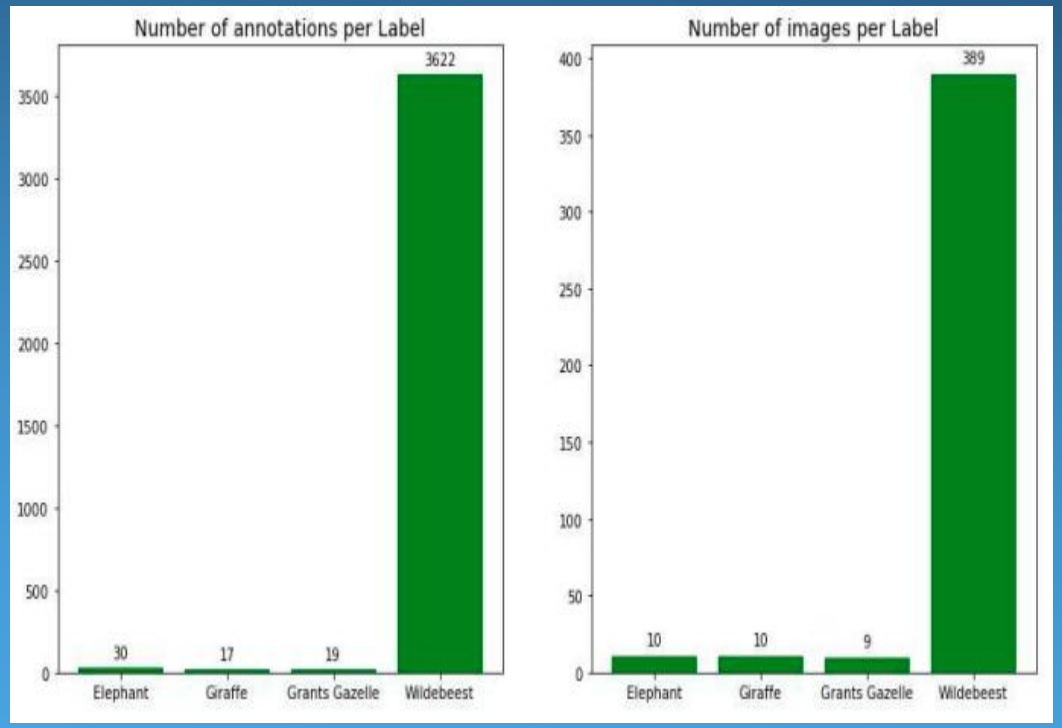
Data used for the project
Subset of Camera trap dataset:

- 4 classes
- Multiple objects in one image
- Diversity of object sizes: large, medium and many small objects
- High resolution: 2592x1944 or 3264x2448
- Highly imbalance

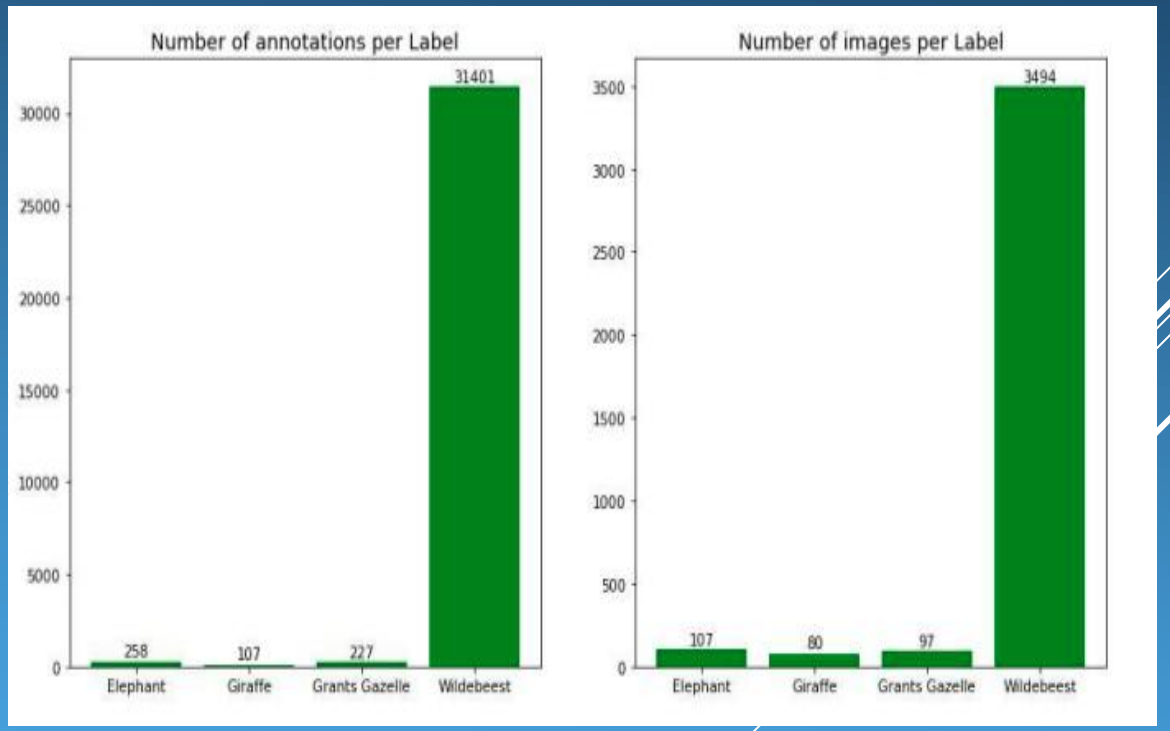


3. Data, Class Imbalance and Few-Shot learning

Train and test sets are split into one to nine ratio
Class imbalance still remains in training and test set



Data distribution - training set



Data distribution - test set

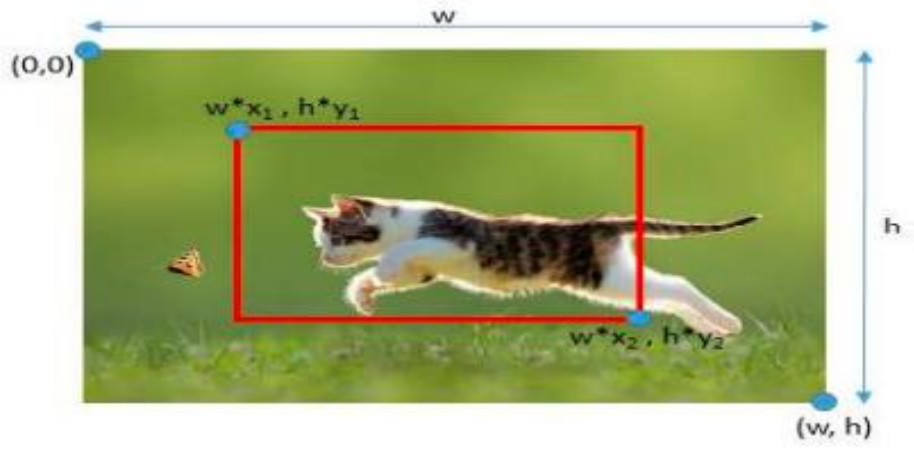
4. Results

Class imbalance:
Trained model and evaluate it on test set, the results is surprised poor. Strongest signal is at Wildebeest class, the dominant class in the dataset

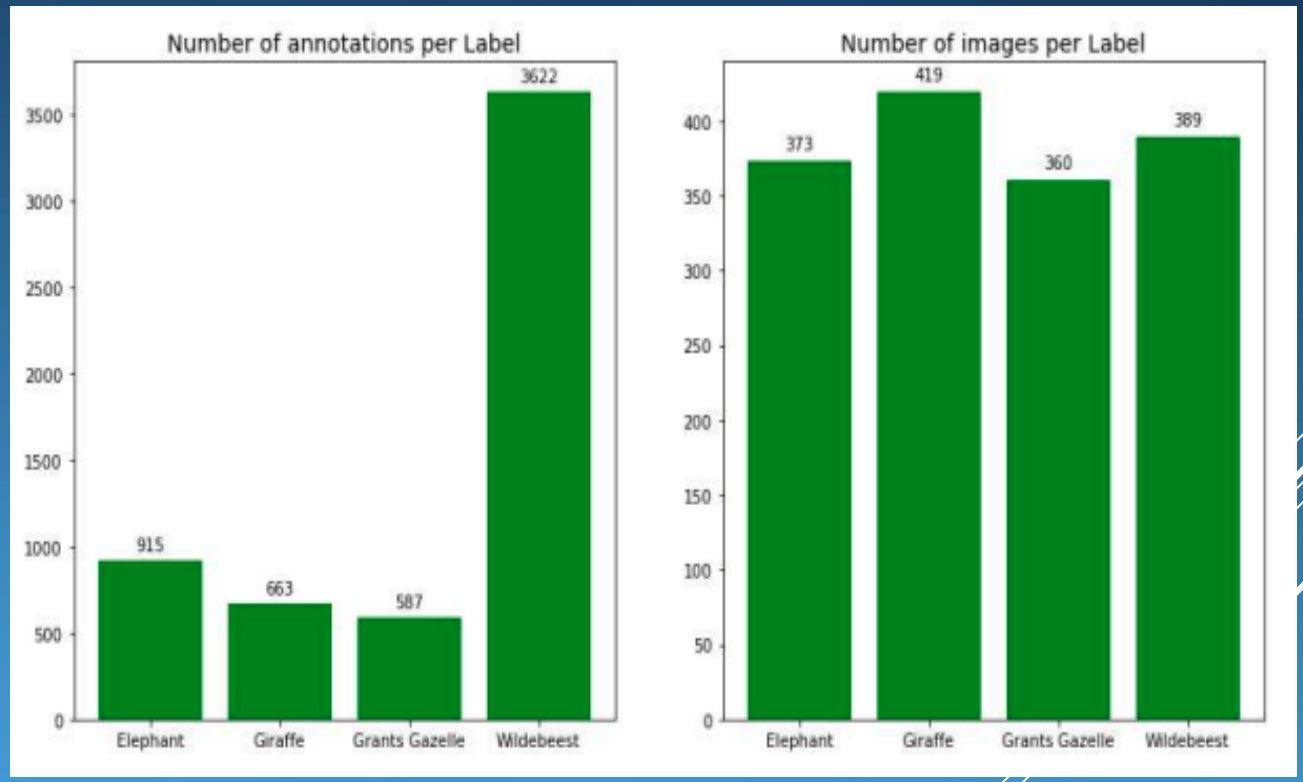
Class	AP
Elephant	0.0011
Giraffe	0.0811
Grants Gazelle	0.0000
Wildebeest	0.1741
Mean Average Precision(mAP)	0.0641

This poor performance is caused by strong class imbalanced. We use image augmentation technique to address this issue.
Using relative coordinates to the width and height of images, we crop 48 images of each image in training set. Locations of cropped images are identified by vector (x_1, y_1, x_2, y_2):
(i/20, 0, 1 1), (0, 0, 1-1.2*(8-i)/20, 1), (i/17, i/25, 1, 1), (0, 0, 1-(8-i)/20, 1-(8-i)/25), (i/30, i/35, 1-(8-i)/30, 1-(8-i)/35) , (i/20, i/20, i/20+0.5, i/20+0.5), for i in {0, 1, 2, 3, 4 ,5 ,6, 7}

4. Results



Class	AP	
	Original	After augmentation
Elephant	0.0011	0.3510
Giraffe	0.0811	0.2444
Grants Gazelle	0.0000	0.5054
Wildebeest	0.1741	0.4304
Mean Average Precision(mAP)	0.0641	0.3828

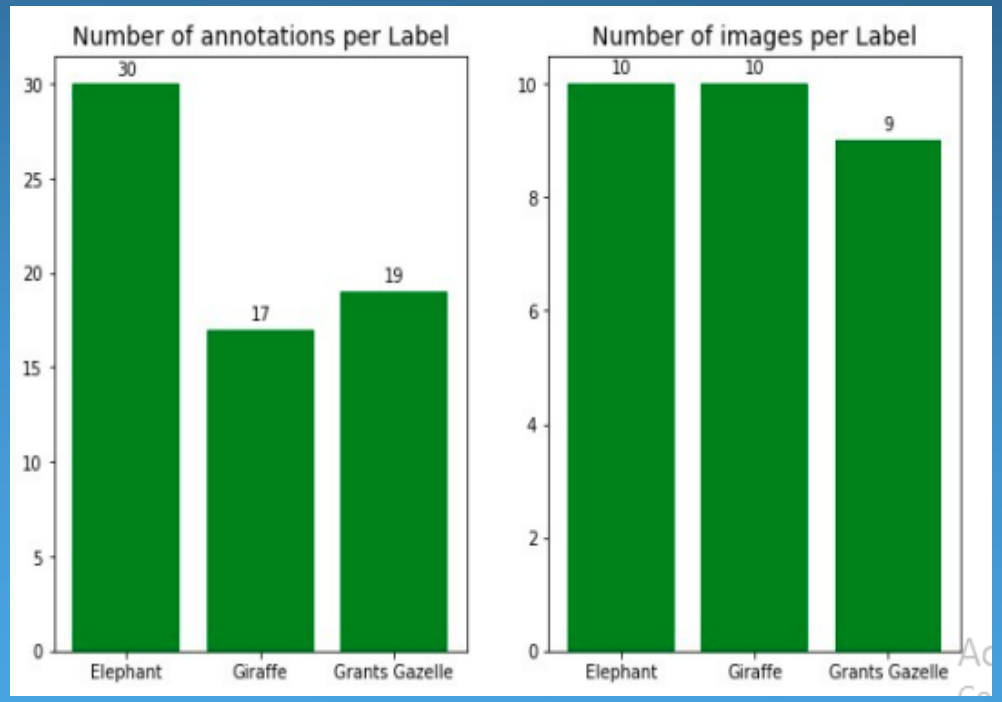


Class imbalance:

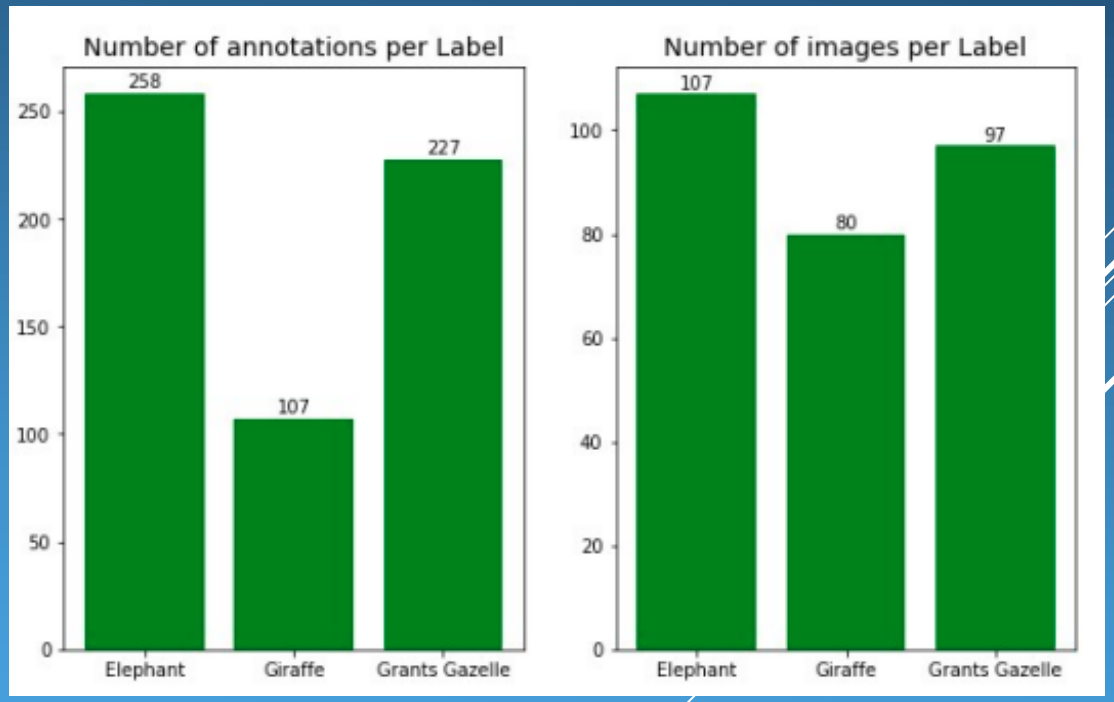
New training set is more balanced in number of images per class. Number of annotations per label also improves. Model performance improve from 0.06 to 0.38mAP

4. Results

Few-shot learning:
The same data in class imbalance but remove Wildebeest class. Number of images per class in training set is limited at 10.



Data distribution in training set



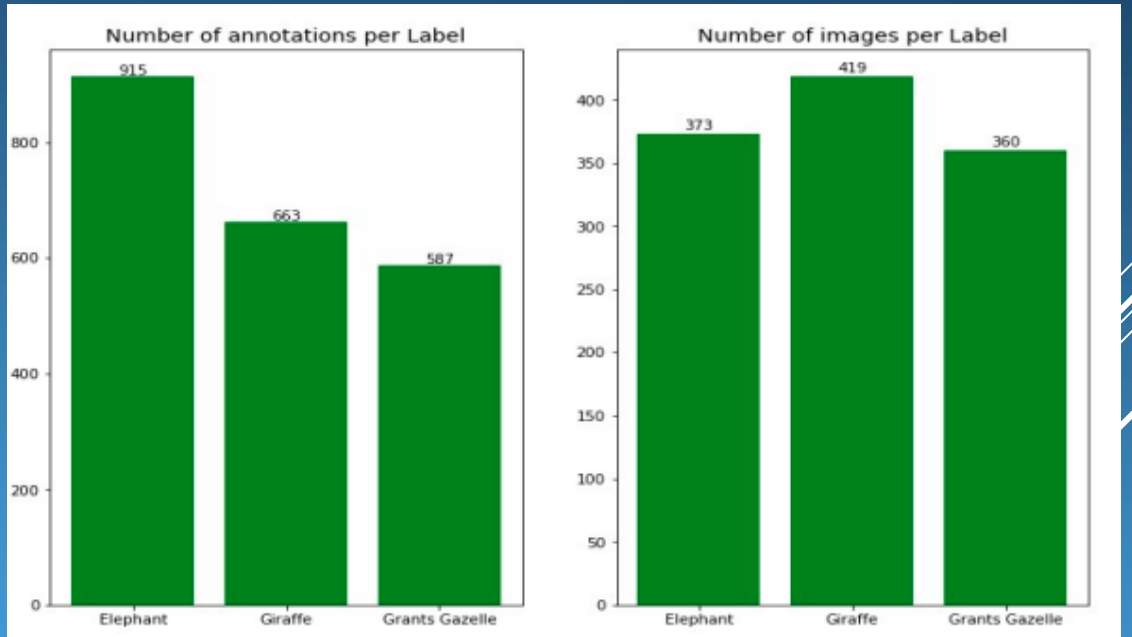
Data distribution in test set

4. Results

Few-shot learning:
Training model and evaluate it on test set, we get a result at 0.114 mAP

Class	AP
Elephant	0.0281
Giraffe	0.0266
Grants Gazelle	0.2882
Mean Average Precision(mAP)	0.1143

Using the same image augmentation technique in class imbalance class to address this issue. The performance raise from just at 0.11mAP to 0.67mAP



Class	AP	
	Original set	Augmented set
Elephant	0.0281	0.6270
Giraffe	0.0266	0.6311
Grants Gazelle	0.2882	0.7739
Mean Average Precision(mAP)	0.1143	0.6773

5. Conclusion and Discussions

Conclusion:

Image augmentation is simple but effective to deal with both class imbalance and few-shot learning. This technique boosts the performance of the model from just 0.06 to 0.38mAP for class imbalance data and from 0.11 to 0.67mAP.

Training time and computational cost also raise significantly when use this method.

Limitation:

- YOLO3 only predicts up to three objects in the same cell.
- Two objects are assigned to the same anchor box then only one is detected

Future work:

- Research newer models: YOLO4, YOLO5, YOLO6
- Generative Adversarial Network (GANs) for generating new images
- Meta-learning algorithms for Few-Shot learning: Siamese Networks, Matching Networks, Meta-Networks, Cross Domain FSL…